

Technical Requirements & Architecture Document

Project: AI-Powered Case Preparation Platform

Role: Solution Architect

1. Frontend

Component	Technology	Purpose
Framework	React.js	Modern component-based UI
Language	JavaScript (ES6+)	Rapid frontend development
Styling	Tailwind CSS	Responsive interface
State Management	React Context / Redux Toolkit	Application state
API Integration	Axios / Fetch API	REST communication

2. Backend

Component	Technology	Purpose
Language	Python	Excellent AI ecosystem
Framework	FastAPI (Primary)	High-performance REST APIs
Alternative	Django	Admin portal and ORM-heavy applications
Authentication	JWT + OAuth2	Secure authentication and authorization

Recommendation: Use FastAPI as the primary backend framework due to its asynchronous support, automatic API documentation, and seamless integration with AI services.

3. Database Layer

3.1 Structured Database

- Google Cloud SQL (PostgreSQL)

Used for:

- User accounts
- Authentication & authorization
- Session scheduling
- Bookings
- Audit logs

3.2 NoSQL Database

- MongoDB Atlas

Used for:

- Chat history
- AI conversations
- Prompt templates
- User preferences
- Case repository metadata
- AI evaluation results

3.3 Vector Database

- Vertex AI Vector Search (Preferred)

- Alternatives: Qdrant, Milvus, Weaviate

Purpose:

- Semantic search
- Retrieval-Augmented Generation (RAG)
- Document embeddings

4. Cloud Infrastructure (Google Cloud Platform)

Resource	Purpose
Cloud Run	Initial backend deployment
Cloud Storage	Store PDFs, documents, uploads
Cloud SQL	Managed PostgreSQL database
Vertex AI	LLM inference and AI services
Secret Manager	Secure API keys and credentials
Cloud Logging	Centralized logs
Cloud Monitoring	Application monitoring
IAM	Role-based access control

5. Deployment Strategy

Phase 1 – MVP

- Frontend: React.js
- Backend: FastAPI deployed on Google Cloud Run
- Database: Cloud SQL + MongoDB Atlas
- CI/CD: GitHub Actions + Docker

Phase 2 – Production

Option A: Google Kubernetes Engine (GKE)

- Auto scaling
- Load balancing
- Rolling updates
- High availability

Option B: On-Premises Deployment (IIM Lucknow)

- Kubernetes cluster hosted within campus infrastructure
- Campus network access
- Hybrid architecture with Vertex AI hosted on GCP

6. AI & Chatbot Architecture

LLMs

Primary Model: Vertex AI - Gemini 2.5 Pro

- Case solving
- Answer evaluation
- Personalized feedback
- Framework generation

Secondary Model: Gemini 2.5 Flash

- Low-latency chat
- Hints
- Conversational support

Open-Source Models (Future)

- Llama 3.x
- Mistral
- Qwen
- Gemma

Chat Memory & Context Management

Short-Term Memory

- Store recent conversation history in MongoDB.

Long-Term Memory

- User preferences
- Interview history
- Performance analytics

Knowledge Base

- Store case PDFs in Cloud Storage.
- Generate embeddings and index them in Vertex AI Vector Search.

RAG Pipeline

User Query → Embedding Search → Retrieve Relevant Documents → Gemini → Response

7. Security

- JWT Authentication
- OAuth2
- HTTPS
- Google Secret Manager
- Role-Based Access Control (RBAC)
- API Rate Limiting
- Input Validation

8. CI/CD

- GitHub Repository
- GitHub Actions
- Docker
- Google Cloud Run (MVP)
- Google Kubernetes Engine (Production)

9. Final Architecture Summary

Layer	Technology
Frontend	React.js + JavaScript
Backend	Python + FastAPI
API	REST APIs
SQL Database	Google Cloud SQL (PostgreSQL)
NoSQL Database	MongoDB Atlas
Vector Database	Vertex AI Vector Search
AI Models	Gemini 2.5 Pro, Gemini 2.5 Flash, Llama, Mistral, Qwen, Gemma
Cloud	Google Cloud Platform
Initial Deployment	Google Cloud Run

Production Deployment	Google Kubernetes Engine / On-Premises Kubernetes
File Storage	Google Cloud Storage
Authentication	JWT + OAuth2
Monitoring	Cloud Logging + Cloud Monitoring
Secrets	Google Secret Manager
CI/CD	GitHub Actions + Docker